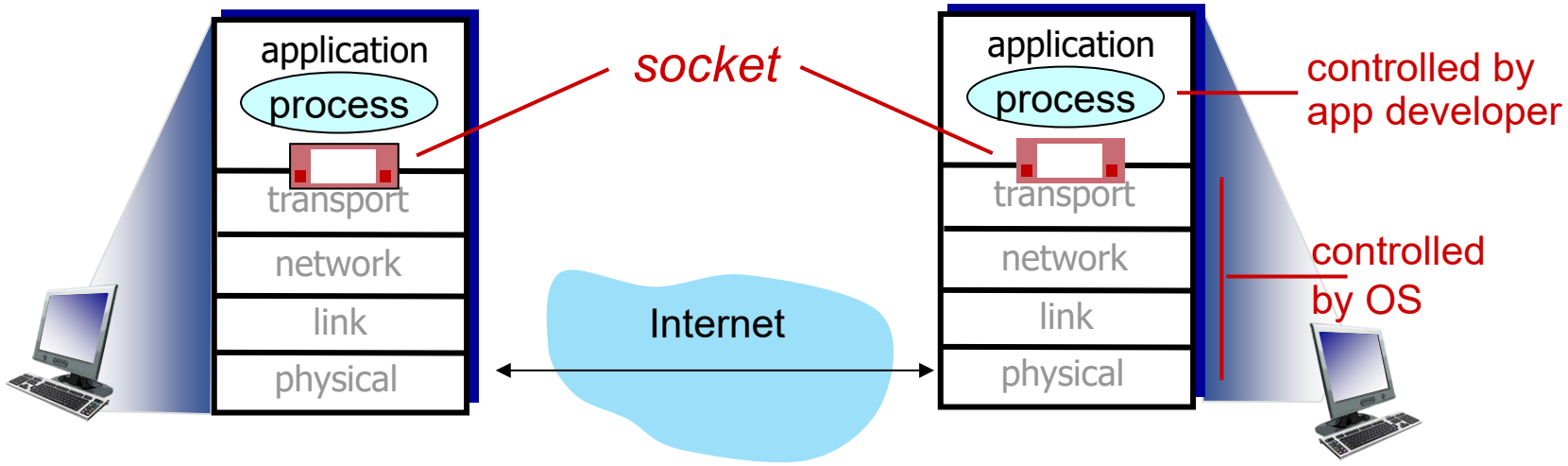
Lecture 10 – Part 1

- Socket Programming

Socket programming

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Socket programming

Two socket types for two transport services:

- *UDP:* unreliable datagram
- *TCP:* reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming with UDP

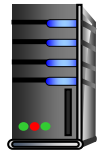
UDP: no “connection” between client & server

- ***no handshaking*** before sending data
- sender ***explicitly attaches*** IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client



create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with server IP and
port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Example app: UDP client

Python UDPClient

include Python's socket library →

```
from socket import *
```

Loopback address (local address)
We will need to use remote IP.

```
server_name = '127.0.0.1'
server_port = 12000
server_address = (server_name, server_port)
```

AF_INET➔ IPv4
SOCK_DGRAM ➔ UDP
AF_INET6➔ IPv6

create UDP socket for client →

```
client_socket = socket(AF_INET,
                       SOCK_DGRAM)
```

get user keyboard input →

```
message = raw_input('Input lowercase sentence:')
```

attach server name, port to message; send into socket →
With UDP: need to specify the destination every time we send data

```
client_socket.sendto(message.encode(), server_address)
```

This waits for response. Reads reply characters from socket into string. 2048: buffer size (bytes)
Blocking call, by default. →

```
modified_message, server_address =
                       client_socket.recvfrom(2048)
```

print out received string and close socket →

```
print modified_message.decode()
client_socket.close()
```

Example app: UDP server

Python UDPServer

```
from socket import *
server_port = 12000
server_socket = socket(AF_INET, SOCK_DGRAM)
server_socket.bind(('', server_port))
print ("The server is ready to receive")
while True:
    message, client_address = server_socket.recvfrom(2048)
    modified_message = message.decode().upper()
    server_socket.sendto(modified_message.encode(),
                         client_address)
```

create UDP socket →

bind socket to local port number 12000 →

loop forever →

Read from UDP socket into message, getting client's address (client IP and port) →

send upper case string back to this client →

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

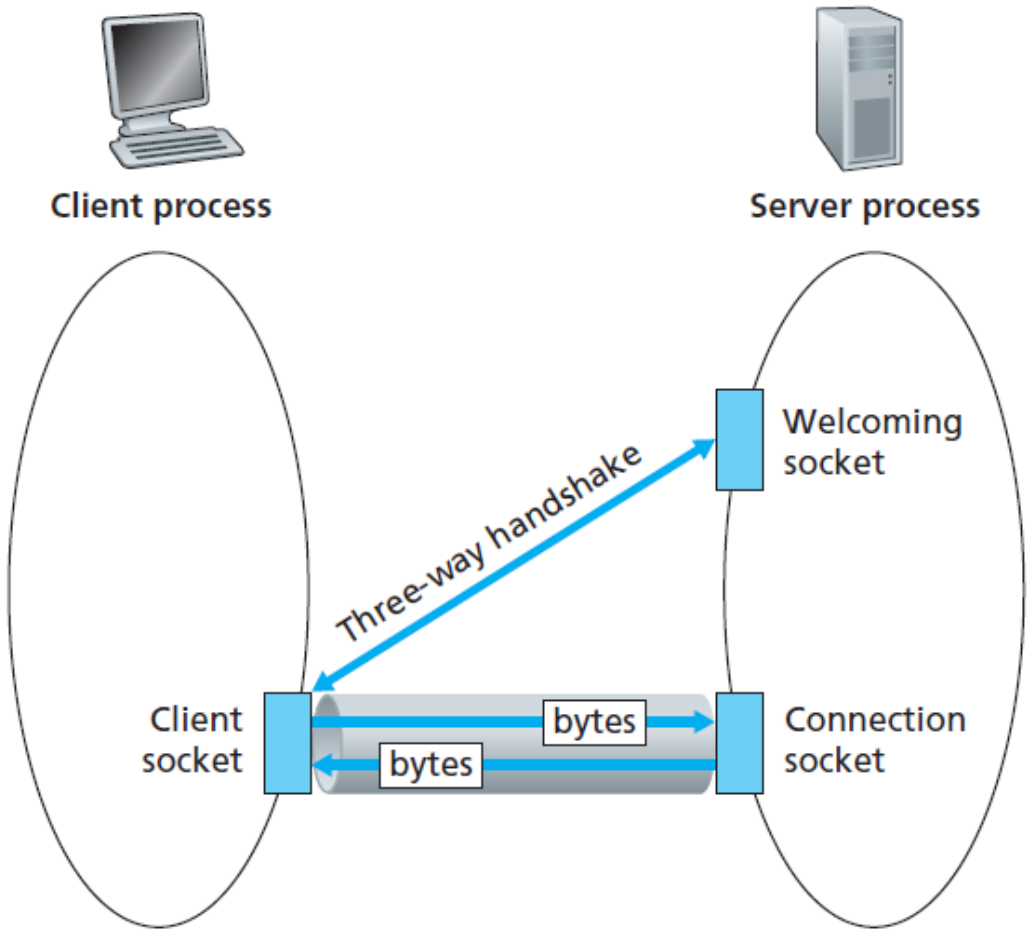
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
  - allows server to talk with multiple clients
  - source port numbers used to distinguish clients (more in Chap 3)

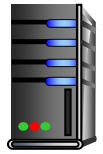
Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server

Welcoming Socket

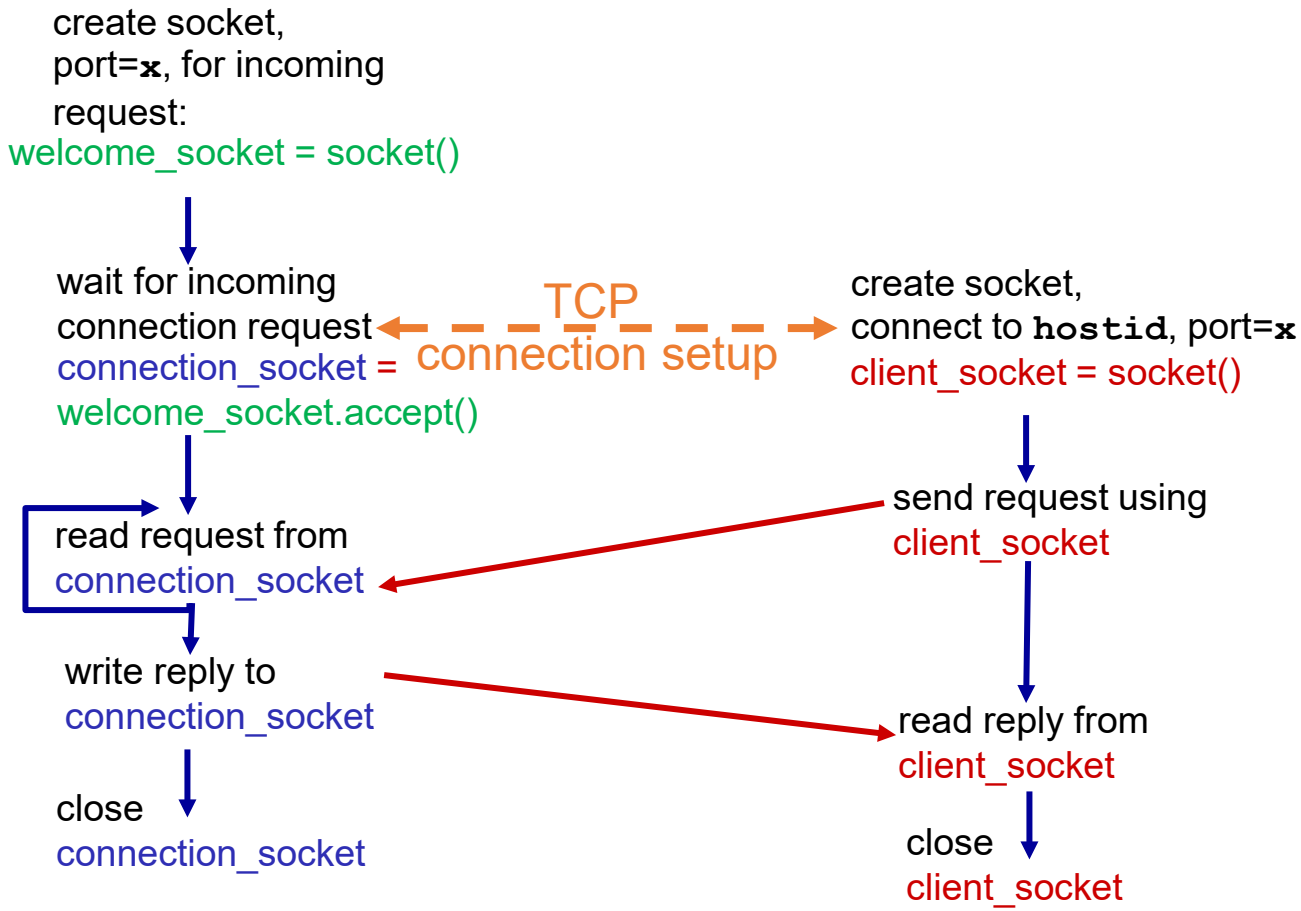


Client/server socket interaction: TCP



server (running on hostid)

client 



Example app: TCP client

Python TCPClient

```
from socket import *
server_name = '127.0.0.1'
server_port = 12000
server_address = (server_name, server_port)
client_socket = socket(AF_INET, SOCK_STREAM)
client_socket.connect (server_address )
sentence = raw_input('Input lowercase sentence:')
client_socket.send(sentence.encode())
modified_sentence = client_socket.recv(1024)
print ('From Server:', modified_sentence.decode())
client_socket.close()
```

create TCP socket for server, remote port 12000

No need to attach server name, port

Example app: TCP server

Python TCPServer

```
from socket import *
server_port = 12000
server_welcome_socket = socket(AF_INET,SOCK_STREAM)
server_welcome_socket.bind((' ',server_port))
server_welcome_socket.listen(1)
print 'The server is ready to receive'
while True:
    connection_socket, client_addr = server_welcome_socket.accept()

    sentence = connection_socket.recv(1024).decode()
    capitalized_sentence = sentence.upper()
    connection_socket.send(capitalized_sentence.
                                        encode())
    connection_socket.close()
```

create TCP welcoming socket →

server begins listening for incoming TCP requests →

loop forever →

server waits on accept() for incoming requests, new socket created on return →

read bytes from socket (but not address as in UDP) →

close connection to this client (but *not* welcoming socket) →

Chapter 2: Summary

our study of network application layer is now complete!

- application architectures
  - client-server
  - P2P
- application service requirements:
  - reliability, bandwidth, delay
- Internet transport service model
  - connection-oriented, reliable: TCP
  - unreliable, datagrams: UDP
- specific protocols:
  - HTTP
  - SMTP, IMAP
  - DNS
  - P2P: BitTorrent
- video streaming, CDNs
- socket programming: TCP, UDP sockets

Chapter 2: Summary

Most importantly: learned about *protocols*!

- typical request/reply message exchange:
  - client requests info or service
  - server responds with data, status code
- message formats:
  - *headers*: fields giving info about data
  - *data:* info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer
- "complexity at network edge"